

Owner B 新手闭卷指南：page784 从 vLLM 调用到 Triton 实现

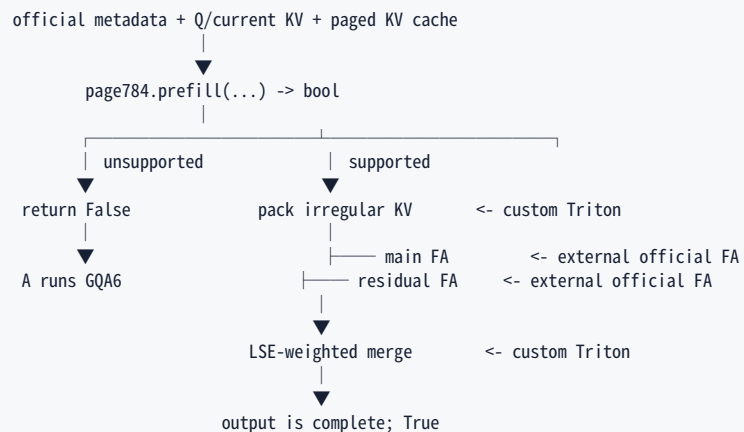
目标：读完后，你应该能不看原实现，独立写出一个逻辑正确、接口兼容、结构可以不同的 page784 fast path；也能解释它如何进入 wheel、何时被调用、Triton 何时编译以及结果为何正确。

主要源码： [vllm/v1/attention/ops/rocm_aiter_unified_attention_page784.py](#)

上游调用： [vllm/v1/attention/backends/rocm_aiter_unified_attn.py](#)

0. 先记住这张“小抄”

你负责的不是完整 attention，也不是 FlashAttention 源码。你负责的是：



闭卷时只背一句：

先判门，按 768 主区和剩余区分割 key/value，各算一次 attention，用 LSE 正确合并。

1. 它怎样在 vLLM 中被调用并生效

1.1 从模型到你的函数

实际调用关系可以压缩为：

```
Qwen3NextAttention
-> vLLM Attention layer
-> RocmAiterUnifiedAttentionImpl.forward(...)
-> common gate
  |___ miss -> official AITER unified_attention
  |___ hit
    -> page784.prefill(...)
      |___ True -> return output immediately
      |___ False -> gqa6.prefill(...) -> return output
```

`rocm_aiter_unified_attn.py` 在模块导入时执行：

```
from vllm.v1.attention.ops import rocm_aiter_unified_attention_page784 as page784
```

在 `forward()` 中先做公共 gate，然后调用：

```
if page784.prefill(query, key, value, key_cache, value_cache,
                  output, attn_metadata, self.scale):
    return output
gqa6.prefill(...)
return output
```

所以你对上游只有一个接口：

```
prefill(...) -> bool
```

- 返回 `False`：我不支持本次输入，而且 `output` 必须完全没变；A 会接着跑 GQA6。
- 返回 `True`：我已经把完整 attention 结果写进 `output`；A 立即返回，不能再算一次。

这个 bool 不是“是否成功”的普通状态码，而是输出所有权交接协议。

1.2 公共 gate 和你的专用 gate

宿主 A 已检查以下条件，你可以把它们当作调用前置条件，但修改这些假设时必须同步修改 A：

- gfx936;
- query heads = 24, KV heads = 4, head size = 256;
- BF16;
- decoder、单请求 prefill;
- cache 单页形状为 [784, 4, 256];
- query/output 连续;
- 没有 ALiBi、sliding window、sink、output quantization 等未支持特性。

你的 `prefill()` 再检查 page784 自己才知道的条件：

```
current_key/current_value exist
128 <= query_len <= 4096
context_len >= 784
single request: query_start_loc.numel() == 2
num_actual_tokens == query_len
packed_pages <= 160
```

必须遵守的顺序是：

```
read scalars -> finish every reject check -> allocate/view/launch/write output
      ▲
      |—— all False returns stay on the left
```

不能先启动 pack 或 merge，再发现不支持而返回 False。否则 GQA6 会接着使用一个已被部分改写的 output，错误会非常隐蔽。

1.3 一次请求中何时更容易命中

page784 面向 later prefill：已有 context 至少 784 token，同时本轮又输入至少 128 个 query token。短 context、单 token decode 和不符合固定头配置的层都不会命中。

这条优化只在命中时生效。源码存在、wheel 安装成功、Triton 编译成功，都不等于运行时一定走到了它；确认生效需要同时验证 gate 和结果。

2. 构建、安装、导入和 JIT：四件不同的事

2.1 wheel 构建做了什么

`python setup.py ... bdist_wheel` 会：

1. 把 `vllm/.../*.py`（包括你的 page784 Python/Triton DSL 文件）打进 wheel；
2. 按 ROCm 构建 vLLM 自己需要的 native 扩展；
3. 生成一个可安装的 `vllm-*.whl`。

它不会提前把 `_pack_page784` 和 `_merge_page784` 为所有运行形状编译成 GPU 二进制。这两个函数有 `@triton.jit`，通常在第一次真实 launch 时按设备、常量参数等进行 JIT。

2.2 `--target` 安装为什么有用

```
python3 -m pip install --no-deps --target "$SITE_DIR" vllm-*.whl
PYTHONPATH="$SITE_DIR" python3 ...
```

`--target` 把新 wheel 解包到隔离目录，`PYTHONPATH` 又让 Python 优先从该目录导入 vLLM。这样不会覆盖比赛容器原来的依赖。

必须检查实际导入位置，防止“构建了新 wheel，却运行了旧源码”：

```
PYTHONPATH="$SITE_DIR" python3 - <<'PY'
import inspect
from vllm.v1.attention.ops import rocm_aiter_unified_attention_page784 as p
print(inspect.getfile(p))
PY
```

路径应指向 `$SITE_DIR/vllm/...`，而不是系统 `site-packages` 或另一个 `checkout`。

2.3 外部 FlashAttention wheel 做了什么

本文件导入：

```
from flash_attn import varlen_fwd_unified
```

它来自比赛容器预装的 DCU FlashAttention wheel。两次主要 attention 由它完成。你只修改 page784 Python/Triton 文件时：

- 要重新构建/安装 vLLM wheel，确保新 Python 文件被部署；
- 不需要重新编译 FlashAttention；
- 不要从公开 PyPI 安装 CUDA 版 wheel 覆盖比赛环境版本。

2.4 Triton 第一次运行做了什么

```
Python imports @triton.jit function
    | no GPU binary yet
    ▼
first launch with device + constexpr + dtype
    |
    ▼
Triton specializes and compiles kernel
    |
    ▼
binary/cache written under TRITON_CACHE_DIR
    |
    ▼
kernel launches; later matching launches reuse cache
```

因此第一次专项测试可能比后续慢。改变 kernel 源码、设备、dtype 或参与特化的常量，可能生成新的缓存条目。TRITON_CACHE_DIR 应设置到明确的可写目录，方便确认新 kernel 确实生成。

2.5 从修改到生效的可靠流程

以下命令属于目标比赛容器操作手册。本文编写环境设备不匹配，未执行 wheel 全量构建、Triton kernel、DCU 或服务实验；已有正确性结果以原 Owner B 文档记录为准。

```
# 1. 静态错误尽早失败
ruff check vllm/v1/attention/ops/rocm_aiter_unified_attention_page784.py
python3 -m py_compile \
    vllm/v1/attention/ops/rocm_aiter_unified_attention_page784.py

# 2. 在临时 worktree 构建 wheel, 避免 setup.py 改写工作源码
git worktree add --detach /tmp/qwen35-build-source HEAD
mkdir -p /tmp/qwen35-build/{dist,bdist,cache}
cd /tmp/qwen35-build-source
VLLM_TARGET_DEVICE=rocm MAX_JOBS=16 python3 setup.py \
    build --build-base /tmp/qwen35-build/build \
    bdist_wheel --bdist-dir /tmp/qwen35-build/bdist \
    --dist-dir /tmp/qwen35-build/dist

# 3. 隔离安装, 不覆盖容器环境
python3 -m pip install --no-deps --target /tmp/qwen35-build/site \
    /tmp/qwen35-build/dist/vllm-*.whl

# 4. 用新 site + 独立 Triton cache 触发首次 JIT 和专项验证
cd /path/to/your/repo
RUN_DIR="$(mktemp -d /tmp/qwen35-b-check.XXXXXX)"
(cd "$RUN_DIR" && HIP_VISIBLE_DEVICES=0 \
    TRITON_CACHE_DIR=/tmp/qwen35-build/cache/b-triton \
    PYTHONPATH=/tmp/qwen35-build/site \
    python3 /path/to/your/repo/docs/cscv/verify_qwen35_optimizations.py page784)
```

注意：临时 worktree 取的是 HEAD。如果改动还未提交，它不会自动出现在该 worktree 中。闭卷比赛里最常见的“新代码没生效”原因之一，就是构建源并不包含刚才的修改。

3. 优化究竟是什么

3.1 普通 attention 的数学

对每个 query，attention 先算分数，再 softmax，再加权 value：

```
score_i = q dot k_i * scale
Z       = sum_i exp(score_i)
weight_i = exp(score_i) / Z
output  = sum_i weight_i * v_i
LSE     = log(Z) = logsumexp(scores)
```

Q 有 24 个头，K/V 只有 4 个头，所以每个 KV 头服务 6 个 query 头。这就是 GQA 6:1。

3.2 为什么把 784 拆成 768 + 16

KV cache 的物理 page 是 784 token，但 768 是更规则的主体，最后 16 token 是小尾巴。对于已有 context：

```
context_len = full_pages * 784 + boundary_tokens
0 <= boundary_tokens < 784
```

每个完整 page 拆成：



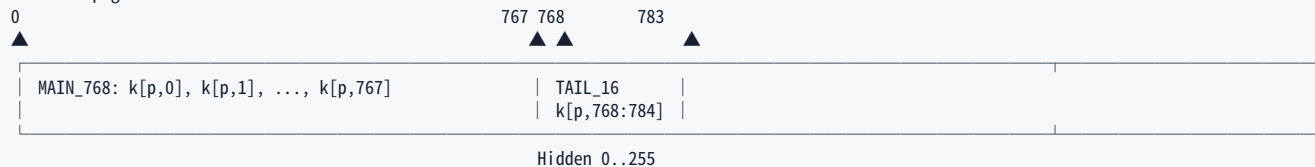
所有 MAIN_768 保持 paged cache 形式，交给一次 non-causal FA。所有 TAIL_16、不足一页的 boundary，以及本轮 current K/V，按逻辑顺序打成 page64，交给另一次 causal FA。

3.3 数据分区图

下面使用最后两个有意义的轴。实际张量还有 KV head 轴 0..3；Hidden/head-dim 为 0..255。图按可读性压缩，坐标不是像素比例。

Tensor: key_cache logical history, shape=[P, 784, 4, 256]
 Formula: page[p] = MAIN[p, 0:768] concat TAIL[p, 768:784]

Token-in-page axis



Tensor: main KV, logical shape=[full_pages*768, 4, 256]
 Formula: main = concat_p key_cache[block_table[p], 0:768]

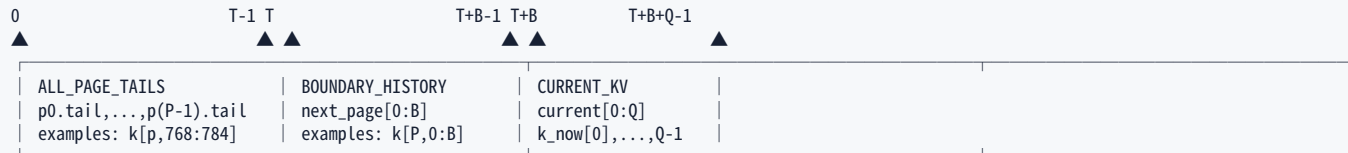
Main token axis



M = full_pages * 768

Tensor: packed residual KV, shape=[packed_pages, 64, 4, 256]
 Formula: residual = all_tails concat boundary concat current_KV

Residual token axis



T=P*16, B=boundary_tokens, Q=query_len

两部分不重不漏：

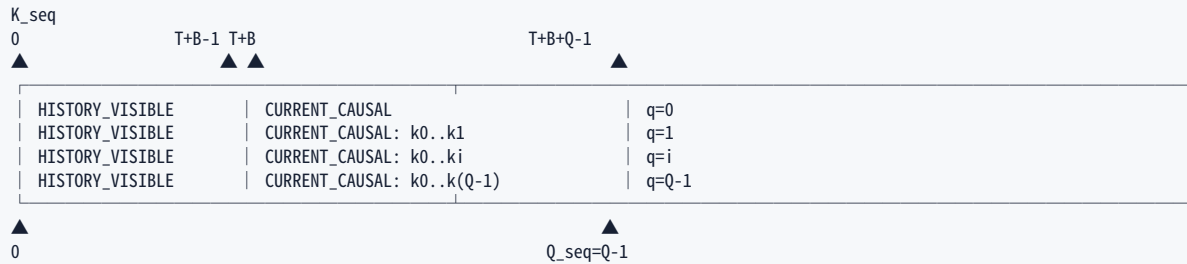
```
main history count    = P * 768
residual history count = P * 16 + B
sum                   = P * 784 + B = context_len
```

3.4 为什么 main non-causal、residual causal

main 里只有本轮 query 之前的历史 token，所以每个 query 都能看全部 main，使用 `causal=False`。

residual 的末尾包含本轮 current K/V。query `i` 只能看到 current 的 `0..i`，不能看到未来的 `i+1..Q-1`，所以使用 `causal=True`。FlashAttention 在 `K_len > Q_len` 时把因果区域 右下对齐，前面的 residual history 对所有 query 可见。

Tensor: residual attention visibility, shape=[Q, T+B+Q]
 Formula: $\text{visible}(q_i, k_j) = (j < T+B) \text{ or } (j \leq T+B+i)$



3.5 为什么不能把两次输出简单平均

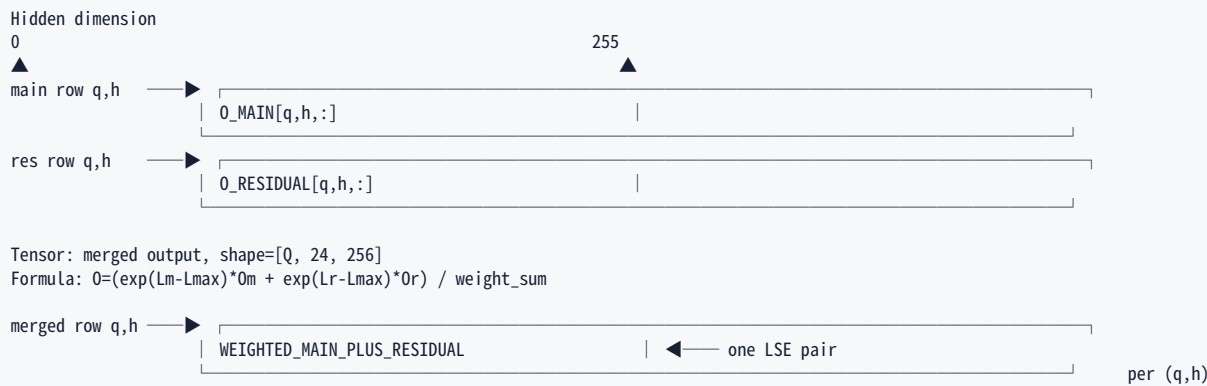
设 main 和 residual 各自 softmax 后得到 0_m 、 0_r ，两边归一化分母分别为 Z_m 、 Z_r 。完整 attention 是：

$$0 = (Z_m * 0_m + Z_r * 0_r) / (Z_m + Z_r)$$

FA 返回 $L_m = \log(Z_m)$ 和 $L_r = \log(Z_r)$ 。为了避免直接 $\exp(L)$ 溢出，先减最大值：

```
L_max = max(L_m, L_r)
w_m = exp(L_m - L_max)
w_r = exp(L_r - L_max)
0 = 0_m * w_m / (w_m + w_r) + 0_r * w_r / (w_m + w_r)
```

Tensor: partial outputs, each shape=[Q, 24, 256]
 Formula: $0_{\text{main}} = \text{softmax}(S_{\text{main}})V_{\text{main}}$; $0_{\text{res}} = \text{softmax}(S_{\text{res}})V_{\text{res}}$



普通平均隐含假设 `Z_m == Z_r`，通常不成立，因此会算错。

4. 读懂 `_pack_page784`：你真正需要的 Triton 入门

4.1 Triton kernel 不是“逐行执行的 Python”

```
@triton.jit
def kernel(...):
    ...

kernel[grid](..., num_warps=4)
```

- `@triton.jit`：函数体用 Triton DSL 描述 GPU kernel。
- `kernel[grid](...)`：配置并启动很多 program instance。
- `tl.program_id(axis)`：当前 program 在 grid 某一轴的编号。
- `tl.arange(0, 256)`：一个 program 内并行处理 256 个 lane/元素。
- `tl.load/tl.store`：按指针和 mask 读写显存。
- `tl.constexpr`：编译期常量，可用于特化和展开。

不要把一个 Triton program 等同于一个 GPU thread。这里可以把它理解为“一小组线程共同执行的向量化工作单元”。

4.2 pack 的 grid 如何覆盖数据

启动方式：

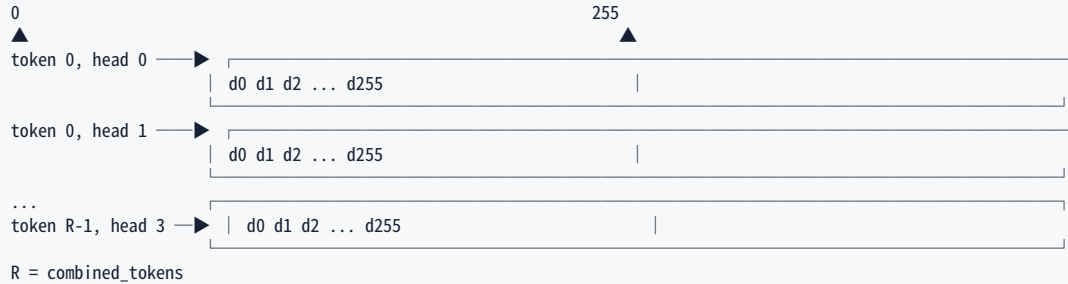
```
_pack_page784[(combined_tokens, 4)](...)
```

因此：

```
grid axis 0 = token in packed residual, 0..combined_tokens-1
grid axis 1 = KV head, 0..3
inside program = head dimension, 0..255
```

Tensor: packed_key_tokens, shape=[combined_tokens, 4, 256] view
 Formula: $\text{dst}[\text{token}, \text{head}, \text{dim}] = \text{selected_source_K}[\text{token}, \text{head}, \text{dim}]$

Head dimension



4.3 token 到源地址的三段映射

先计算：

```
tail_tokens = full_pages * 16
residual_tokens = tail_tokens + boundary_tokens
combined_tokens = residual_tokens + query_len
```

对目标 token：

1. $\text{token} < \text{tail_tokens}$ ：来自完整页尾部； $\text{logical_page} = \text{token} // 16$ ， $\text{position} = 768 + \text{token} \% 16$ 。
2. $\text{tail_tokens} \leq \text{token} < \text{residual_tokens}$ ：来自 boundary； $\text{logical_page} = \text{full_pages}$ ， $\text{position} = \text{token} - \text{tail_tokens}$ 。
3. $\text{token} \geq \text{residual_tokens}$ ：来自 current K/V； $\text{current_token} = \text{token} - \text{residual_tokens}$ 。

Triton 用 `tl.where` 计算候选值，用 load mask 保证只从有效来源读取。

4.4 为什么必须使用 runtime stride

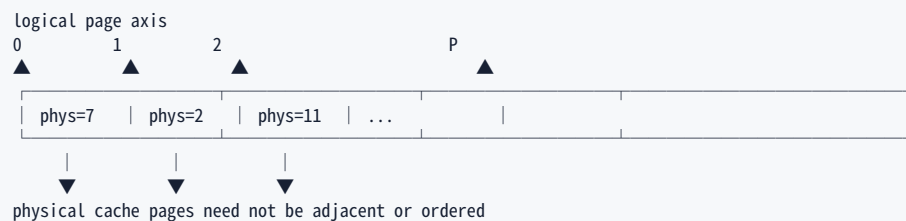
张量 shape 只说明“有多少元素”，stride 才说明“相邻索引在内存里跨多少元素”。交错 view 可能仍显示 `[pages, 784, 4, 256]`，但不能用连续布局公式猜地址。

对 cache：

```
offset = physical_page * stride_page
+ position * stride_token
+ head * stride_head
+ dimension * stride_dim
```

K 和 V 分别传自己的完整 stride；current K/V 也分别传 stride。 `block_table[logical_page]` 把逻辑页号映射到实际 physical page。

Tensor: logical-to-physical page mapping
Formula: `physical_page = block_table[logical_page]`



闭卷时写 offset 的安全方法：不要背一条长公式；按每个索引轴逐项加 `index * runtime_stride`，最后再加 head dimension。

4.5 为什么目标 offset 是 `token * 1024 + head * 256 + dim`

packed view 是连续的 `[token, 4, 256]`：

```
stride_token = 4*256 = 1024
stride_head = 256
stride_dim = 1
```

所以这个公式只用于你自己分配且已知连续的目标 buffer。不能把它照抄到输入 cache。

5. 读懂 `_merge_page`

5.1 grid 与四行特化

输出逻辑形状为 `[query_len, 24, 256]`，共有 `query_len*24` 个 (token, head) 行。一次 program 处理 4 行：

```
row = tl.program_id(0) * 4 + tl.arange(0, 4)
token = row // 24
head = row % 24
```

因此 program 数量：

```
(query_len * 24) / 4 = query_len * 6
```

这就是 launch grid `[(query_len * 6,)]` 的来源。它依赖 query heads 恰好是 24，而这个条件 由宿主公共 gate 保证。

每行再用 `tl.arange(0,256)` 并行覆盖 head dimension。LSE 对每个 (head, token) 只有一个 标量，而 output 对每个 (token, head) 有 256 个数；该标量广播到整行。

5.2 LSE 的内存顺序

代码使用：

```
lse_offset = head * query_len + token
```

即把 LSE 视为 `[24, query_len]`。如果外部 FA 的版本返回三维 `[1,24,Q]`，调用端先用 `lse[0]` 去掉 batch 轴。修改 FA 调用或版本时必须重新确认 LSE 的 shape/layout，不能只看 数值类型。

5.3 FP32 的意义

LSE 权重和 main/residual value 在 merge 中转为 FP32 计算，可减少 BF16 下指数与加权的 误差。最终写入 BF16 output 时才发生目标 dtype 的舍入。

6. prefill() 逐段理解

6.1 gate：先证明“我能算”

从 metadata 得到：

```
query_len = metadata.max_query_len
context_len = metadata.max_seq_len - query_len
```

完成所有条件检查后才继续。这里依赖单请求，所以 max length 就是本次 request 的真实长度；若未来支持 batch，不能继续这样简化。

6.2 workspace: 复用大内存，不在每层重复申请

缓存 key 是 (query.device, query.dtype)。第一次为该设备和 dtype 分配：

- 两个 [4096,24,256] token buffer: main output、residual output;
- 两个 [160,64,4,256] page buffer: packed K、packed V。

以后返回所需前缀 view，不重建大 tensor。约 136 MiB 是预防性资源上界；首次申请 OOM 没有异常恢复，所以部署前应保证余量。

6.3 metadata: 只缓存会重复使用的小张量

main/residual FA 分别需要：

```
main_len      = [full_pages * 768]
residual_len  = [combined_tokens]
residual_table= [[0, 1, ..., packed_pages-1]]
```

缓存 key 包含 device 以及会改变内容的长度。原始 main block table 直接复用官方 metadata 的完整页前缀；residual block table 是打包 workspace 中连续 page64 的编号。

6.4 两次官方 FA

main 调用的 K/V 是 key_cache[:, :768] / value_cache[:, :768]，page size 因 view 变成 768；block table 使用原表前 full_pages 项。

residual 调用的 K/V 是 [packed_pages,64,4,256]，block table 为连续编号。两次都要求返回 softmax LSE，为随后正确合并做准备。

6.5 merge 和返回

merge 将每个 (query token, query head) 的两个 partial output 合成完整 output。完成后返回 True，上游直接返回该 output。

7. 对上接口：哪些能改，哪些不能偷偷改

7.1 prefill 参数契约

参数	含义	B 如何使用
query	[Q,24,256] BF16	两次 FA 的共同 Q
current_key/value	[Q,4,256]	打包在 residual history 后面
key/value_cache	[physical_pages,784,4,256] view	main 直接读，tail/boundary 按 stride 打包
output	[Q,24,256]	只有命中后才写；True 时必须完整
metadata	官方 FA metadata	长度、query starts、block table、token 数
scale	通常为 $1/\sqrt{256}$	两次 FA 必须相同

7.2 修改这些内容必须与 A 同步

- prefill 函数签名；
- 返回 bool 的语义；
- cache shape/layout/stride 前提；
- metadata 字段语义；
- 24/4/256、BF16、单请求等公共 gate 假设；
- False 时 output 不变的承诺。

只修改 pack 内部写法、workspace 组织或 merge tile，而且不改变上述契约时，通常可在 B 文件 内部独立迭代，但仍必须跑边界、交错 stride 和数值测试。

8. 官方积木拼装策略：少背代码，多背搜索入口

闭卷不等于把 230 行全部背下来。更稳的策略是：把官方 vLLM 当作现场可检索的标准库，只记住 page784 独有的分区公式和接口契约。

8.1 四块官方代码分别解决什么

你要解决的问题	官方原版 vLLM 中搜索什么	可直接借鉴的内容	你仍需自己写的部分
metadata 接口	<code>class FlashAttentionMetadata</code>	字段名、长度和 block table 语义	page784 专用 gate 和长度推导
两段 attention	<code>def cascade_attention</code>	non-causal prefix、causal suffix、两次返回 LSE、最后 merge 的控制流	将 prefix/suffix 改成 main/residual，并准备各自 KV/table
paged cache 地址	<code>physical_block_idx</code> 、 <code>stride_k_cache_0</code>	block table 查物理页、逐轴 runtime stride 计算地址	784 页尾/boundary/current 的三段映射
两路结果合并	<code>def merge_attn_states</code> 、 <code>merge_attn_states_kernel</code>	max-LSE、exp、归一化权重、输出布局	可以先零自定义直接调用官方 helper；性能需要时才写四行/CTA 特化

对应文件：

- 官方 metadata/cascade: `vllm/v1/attention/backends/flash_attn.py`
- 官方 paged attention: `vllm/v1/attention/ops/triton_unified_attention.py`
- 官方 merge dispatch: `vllm/v1/attention/ops/merge_attn_states.py`
- 官方 Triton merge: `vllm/v1/attention/ops/triton_merge_attn_states.py`

本分支说明所对应的官方版本是 vLLM v0.18.1。若比赛现场源码版本不同，优先读现场函数 签名，不要凭记忆强行套旧参数。官方在线入口：

- https://github.com/vllm-project/vllm/blob/v0.18.1/vllm/v1/attention/backends/flash_attn.py
- https://github.com/vllm-project/vllm/blob/v0.18.1/vllm/v1/attention/ops/triton_unified_attention.py
- https://github.com/vllm-project/vllm/blob/v0.18.1/vllm/v1/attention/ops/triton_merge_attn_states.py

8.2 现场只需记住的搜索命令

```
# 1. metadata 有哪些字段
rg -n "class FlashAttentionMetadata" vllm/v1/attention/backends/flash_attn.py

# 2. 官方怎样做“两次 FA + LSE merge”
rg -n "def cascade_attention" vllm/v1/attention/backends/flash_attn.py

# 3. 官方怎样从 block table 找物理页、怎样传 stride
rg -n "physical_block_idx|stride_k_cache|block_table_stride" \
    vllm/v1/attention/ops/triton_unified_attention.py

# 4. 官方稳定 merge 的完整公式和 layout
rg -n "def merge_attn_states|def merge_attn_states_kernel" \
    vllm/v1/attention/ops/{merge_attn_states,triton_merge_attn_states}.py

# 5. 找当前版本 FA 包装函数真实签名
rg -n "flash_attn_varlen_func|varlen_fwd_unified" \
    vllm/v1/attention flash_attn 2>/dev/null
```

你不必记行号，因为上游升级后行号会变；记“文件 + 符号名 + 它解决的问题”更可靠。

8.3 最省记忆的实现路线

第一版以正确为先：

```
official FlashAttentionMetadata
    |
    ▼
custom 10-line length/gate math
    |
    ▼
custom pack kernel
    ├── address pattern copied from official paged attention
    └── only 3-way token mapping is page784-specific
    |
    ▼
two FA calls shaped like official cascade_attention
    |
    ▼
official merge_attn_states(...)
```

这一版甚至可以不用写 `_merge_page784`。直接导入官方 helper：

```
from vllm.v1.attention.ops.merge_attn_states import merge_attn_states

merge_attn_states(
    output, main, main_lse, residual, residual_lse
)
```

在 ROCm 上，该 dispatcher 会走官方 Triton merge。先用它证明 partition、causal 和 pack 都正确，再用 benchmark 判断是否值得换成当前四行/CTA 特化。当前比赛分支保留自定义 merge 是为了固定形状性能，不是因为官方公式不正确。

同样，第一次实现 metadata 时不要创建自己的 class；直接消费上游传来的官方 FlashAttentionMetadata。你只需派生三个小 tensor：main_len、residual_len 和 residual_table。

8.4 从官方 cascade_attention 改造，而不是从空白写控制流

官方 cascade 的骨架是：



page784 只做这组替换：

官方 cascade 概念	page784 对应物
shared prefix KV	每个 784 page 的 0:768 main
per-query suffix KV	tails + boundary + current 的 residual
prefix length/table	P*768 与原 block table 前 P 项
suffix length/table	P*16+B+Q 与连续 page64 table
prefix non-causal	main non-causal
suffix causal	residual causal
official merge	可直接复用，或最后做形状特化

这样你只需证明“page784 分区怎样映射成官方 cascade 的两段”，不需要重新发明两段 attention 的控制流。

8.5 从官方 paged attention 只抄“地址语法”

不要试图读懂整个 `triton_unified_attention.py`。它很长，而你只需要两件事：

```
physical_page = load(block_table + logical_page)
source_offset = sum(axis_index * tensor.stride(axis))
```

官方代码中的 `physical_block_idx * stride_*_0` 和其余 axis stride，就是你的可信模板。你独有且必须记住的仅是：

```
tail token    -> page=token//16, pos=768+token%16
boundary token -> page=P,          pos=token-tail_tokens
current token  -> current_index=token-residual_tokens
```

也就是说，Triton 中最容易写错的“地址框架”从官方找；真正需要闭卷推导的“三段 token 语义”只有三行。

8.6 从官方 merge 做逐项核对

若最终为了性能写 `_merge_page784`，打开官方 `merge_attn_states_kernel`，逐项打勾：

```
[ ] LSE index is [head, token]
[ ] max_lse = max(main_lse, residual_lse)
[ ] main_exp = exp(main_lse - max_lse)
[ ] residual_exp = exp(residual_lse - max_lse)
[ ] scales are divided by their sum before multiplying output
[ ] output row covers the complete head dimension
[ ] source/output strides or contiguity assumptions are protected by gate
```

不要凭印象重新推导数值稳定细节；这正是最适合直接对照官方代码的部分。

8.7 “官方可抄”与“必须理解”的边界

Memory burden		
0 ▲		high ▲
FIND IN OFFICIAL vLLM	MUST UNDERSTAND / RE-DERIVE	
metadata fields	bool ownership contract	
FA call signature	784 = 768 + 16 partition	
block-table stride idiom	residual ordering	
stable LSE merge	non-causal main vs causal residual	
packaging conventions	all rejection and workspace limits	

闭卷考查的核心不应该是你能否默写官方 API，而是你能否在版本对应的官方代码中迅速找到 API，并保持右半边的不变量。

9. 如何快速学会“正确的闭卷实现”

不要求逐字复现。正确性来自不变量，不来自代码长得一样。

9.1 闭卷前应能写出的十条不变量

1. 所有 reject 都发生在任何 output 写入之前。
2. `context = P*784+B`，其中 $0 \leq B < 784$ 。
3. main 恰好包含每个完整页的 0:768。
4. residual 顺序恰好是所有 768:784、boundary、current。
5. main 与 residual history 不重不漏，合计等于 context。
6. cache 源地址使用 `block_table + runtime stride`。
7. current K/V 使用各自 runtime stride。
8. main FA non-causal, residual FA causal。
9. merge 使用 LSE 权重和 max-shift，不做平均。
10. True 表示 output 完成；False 表示 output 原封不动。

能从这十条重新推导代码，就已经达到“闭卷但不必一模一样”。

9.2 推荐的空白纸实现顺序

```
Step 1 write interface and all gates
Step 2 derive P, B, T, residual_tokens, combined_tokens, packed_pages
Step 3 define workspace shapes and cache keys
Step 4 write pack index mapping on paper
Step 5 translate mapping to program_id/arange/load/store
Step 6 prepare main/residual lengths and block tables
Step 7 call official FA twice with identical scale/options
Step 8 normalize returned LSE rank
Step 9 write stable LSE merge
Step 10 return True and test every boundary
```

先写接口和数学，再写 Triton。不要一上来背 kernel。

9.3 允许不同的实现结构

以下结构差异不必与现实一致，只要性能和测试通过：

- helper 函数如何拆分；
- workspace/metadata cache 封装方式；
- merge 一个 program 处理几行；
- pack 的条件表达式写法；
- 变量名和注释；
- 对 metadata shape 做更多显式断言或保守 fallback。

但不能改变核心 partition、因果性、stride 地址、LSE 合并和 bool 契约。

9.4 最小 Triton 学习练习

按以下顺序练，不要先学完整 Triton 生态：

1. 写连续向量 copy：理解 program_id、arange、mask。
2. 写二维 [row,dim] copy：理解 stride。
3. 写 block-table gather：理解 logical→physical 映射。
4. 写三段来源 pack：理解 tl.where 和 masked load。
5. 写两路向量加权：理解广播、FP32 和二维 offset。

这五步正好覆盖本任务所需知识；矩阵乘、softmax、FA 内核细节不是你的闭卷实现范围。

10. 测试：证明的不是“能跑”，而是每条不变量

10.1 gate 边界

至少验证：

输入	预期
Q=127, context=784	False, output 不变

输入	预期
Q=128, context=783	False, output 不变
Q=128, context=784	True, 数值对齐
Q=128, context=800	True, 含 16 boundary
Q=4096 且其余满足	True
Q=4097	False, output 不变
packed_pages=160	可命中
packed_pages=161	False, output 不变

10.2 pack 顺序

不要只喂随机数。可让每个元素编码自己的来源：

```
value(page, position, head, dim)
    = page*large_constant + position*small_constant + head
```

这样一眼就能发现页号、position 或 head 算错。分别检查：

- 第一页 tail 的第一个/最后一个 token；
- 相邻两页 tail 的交界；
- tail 到 boundary 的交界；
- boundary 到 current 的交界；
- residual 最后一个 token。

10.3 交错 cache

必须构造 shape 相同但 stride 不连续的 K/V view，并打印：

```
print(key_cache.shape, key_cache.stride())
print(value_cache.shape, value_cache.stride())
```

只测连续 [pages, 784, 4, 256] 会让错误的固定 offset 看起来正确。

10.4 数值比较

专项脚本会把 page784 输出与完整 attention reference 比较。除 kernel allclose 外，还要跑 4-8K、8-16K、16-32K 三档完整文本精度，因为局部张量通过不代表路由和服务集成正确。

10.5 确认真的走到新代码

建议按从便宜到昂贵的顺序排查：

1. `inspect.getfile(page784)` 确认导入路径；
2. 检查运行参数是否满足公共 gate 和专用 gate；
3. 使用独立 `TRITON_CACHE_DIR`，确认出现 `_pack_page784`、`_merge_page784` 缓存；
4. 专项脚本确认 `accepted=True`；
5. 最后做服务级冒烟和性能测量。

11. 最常见的错误与定位

错误 1：把 shape 当成地址

症状：连续 cache 正确，交错 cache 错。修复方向：逐轴检查 runtime stride，K/V 和 current K/V 分开处理。

错误 2：residual 顺序错

症状：context=784 可能过，context=800 或多页失败。修复方向：用带来源编码的数据检查三段 边界，不要只看最终 allclose。

错误 3：residual 使用 non-causal

症状：本轮较早 query 看到了未来 current K/V，文本精度异常。修复方向：画出 `[Q,K]` 可见区，确认 current 部分为下三角且 history 全可见。

错误 4：两路 output 直接平均

症状：能运行但误差明显。修复方向：检查两边 LSE 的 shape、索引顺序和 max-shift 权重。

错误 5: False 前已经写 output

症状: fallback 的 GQA6 结果偶发污染。修复方向: 把所有可能 reject 的计算集中到首次 kernel launch 之前, 并用哨兵值验证 output 不变。

错误 6: 以为 wheel 构建已经编译 Triton

症状: 首次运行慢或首次才报 kernel 编译错误。修复方向: 理解 Python 打包与 Triton JIT 是两个阶段, 专项测试必须真实触发 kernel。

错误 7: 测试运行了旧代码

症状: 改动无效果, 缓存也没变化。修复方向: 检查 worktree 是否包含提交、打印模块路径、清楚 PYTHONPATH 优先级。

12. 你应该能口述的完整答案

vLLM 的 ROCm ALTER attention backend 在公共 gate 命中 Q24/KV4/D256、BF16、gfx936、单请求 prefill 等条件后, 先调用我的 `page784.prefill()`。我的专用 gate 再检查 query、context、current KV 和 workspace 上界。若不支持, 我在任何写 output 的操作前返回 False, 上游改走 GQA6。

命中时, 我把每个 784-token cache page 的前 768 token 作为 main; 把每页后 16 token、boundary history 和 current KV 按逻辑顺序用 runtime stride 打包成 page64 residual。main 只有历史, 所以调用 non-causal 官方 FlashAttention; residual 含本轮 KV, 所以调用 causal 官方 FlashAttention。两次调用都返回 output 和 LSE。我用稳定的 max-LSE 权重在 FP32 中合并两路 output, 写入最终 output 后返回 True。

vLLM wheel 构建只会打包这个 Python/Triton DSL 文件并构建 vLLM native 扩展; 两个自定义 Triton kernel 在首次真实 launch 时 JIT 并进入 Triton cache。外部 DCU FlashAttention wheel 不由我重编。验证时我要确认新 wheel 的导入路径、gate 命中、Triton 缓存、连续和交错 stride、边界 fallback 的 output 不变、数值 reference 和最终服务文本精度。

如果你能不看源码写出第 9.1 节十条不变量, 并从它们推导出 pack、两次 FA 和 merge, 你就已经掌握了这项优化, 而不是只记住了当前 230 行代码。

13. 十轮自审记录

以下审查以 Owner B 原说明、当前 page784 源码及官方 vLLM 对应实现为依据；只做静态核对， 未运行全量构建、GPU/Triton 或服务实验。

轮次	检查主题	检查结果与完善
1	职责覆盖	gate、workspace、pack、两次 FA、LSE merge、bool 全覆盖
2	官方模板	增加 metadata、cascade、paged stride、official merge 搜索地图
3	调用接口	明确 True=output完成、False=output不变及 A/B 同步边界
4	分区数学	复算 $P*768 + P*16 + B = context$ ，三段不重不漏
5	因果性	用 [Q,K] 区域图核对 main non-causal、residual causal
6	地址/layout	核对 block table、cache/current 各自 stride 和连续目标 offset
7	LSE 数值	对照官方 merge 核对 [head,token]、max-shift 与 FP32 权重
8	构建/JIT	区分 vLLM wheel、外部 FA wheel、Triton 首次 JIT 和缓存
9	闭卷/验收	压缩十条不变量，覆盖 gate 边界、交错 cache、文本精度
10	可视化/新手性	检查轴起止、区域坐标、平直边框、英文图内标签与口述答案

最终静态结论：本文不要求记忆官方 API 代码，依靠现场符号检索即可重建大部分控制流；只有 page784 分区、三段 pack、因果性和 bool 契约必须真正理解。设备结果以原 Owner B 已记录的 正确文档和目标环境验收为准。